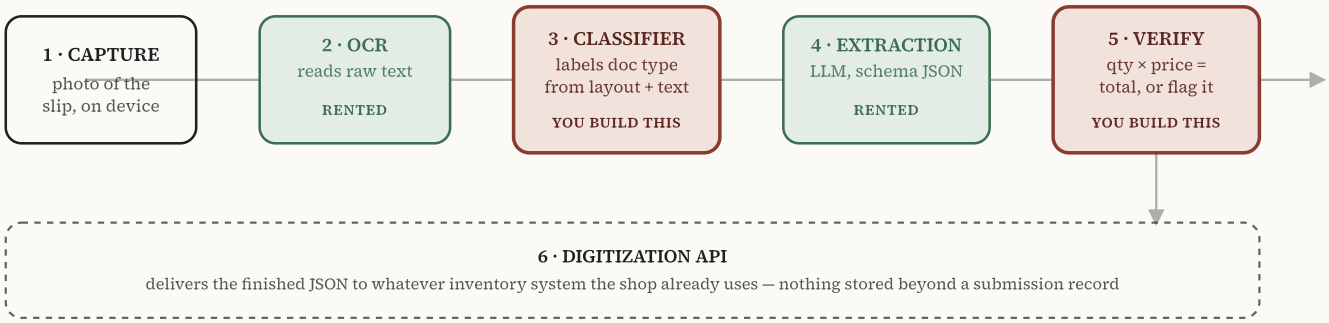
CSE 498A · PRE-DEFENCE PREP

# Viva Prep — Pipeline & Likely Questions

One page, read before tomorrow. First: the pipeline in plain terms, so nobody on the team mixes up who does what. Second: the questions most likely to come from the panel, with the honest answer already written out.

## 1. The pipeline, in order

Six stops. Two are yours to build. Four are rented services you call over the internet and never touch the inside of.




Green = rented service, called over an API, never modified. Terracotta = the two pieces the team designs and owns.

> **Say this out loud tonight, once, all together:** "Photo, then OCR, then our classifier, then the LLM, then our verification check, then the API." In that order. The classifier comes *before* the LLM, not after — the LLM needs to be told the document type before it knows which fields to expect.

## 2. Who owns what — and why the classifier isn't "an LLM skill"

The most common mix-up on the team so far: thinking the classifier is a feature inside the rented LLM. It isn't. It's a separate, smaller model, trained and hosted by the team, sitting on the same backend as everything else except the camera.

**RENTED — CALLED, NEVER TRAINED**

- OCR service (reads the image → raw text)
- LLM (raw text + doc-type label → JSON)

Called over an API, same as any third-party service. No access to their internals, no training involved.

**YOURS — BUILT, TRAINED, HOSTED**

- Document-type classifier
- Arithmetic verification check

Runs on your own server, between OCR and the LLM. Fine-tuned on real documents collected from shops — that's what the corpus is for.

### Why the classifier needs shop data, not a public dataset

Public document datasets are clean, Western, typewritten. Real shop paperwork is thermal receipts, handwritten delivery notes, torn corners, faded ink. A model trained only on clean examples guesses worse on messy real ones. So the corpus exists to teach the classifier what a Bangladeshi shop's actual receipts and delivery notes look like — not to teach it English, not to teach it to read.

### Why raw, messy OCR text is not a blocker

The classifier isn't reading the text alone. It's a layout-aware model — it reads the words *and* where they sit on the page: totals near the bottom, a header block at the top, a grid of line items in the middle. That layout shape survives OCR noise better than the individual words do. That's the actual reason this class of model exists — it was built for exactly this problem, not despite it.

## 3. Likely questions, with the honest answer ready

likely = expected, someone will probably ask this. sharp = harder, technical panel members may dig here. Every answer below is the one already written into the report or worked out in prep — don't improvise a different one live.

LIKELY **How does the LLM get trained?**

It doesn't. The LLM is a commercial model called through an API — never trained by the team. The *classifier* is the one piece that gets trained, and even that is fine-tuning an existing pretrained layout model, not training from zero.

LIKELY **If you're not training the LLM, why collect documents from shopkeepers at all?**

Two separate reasons. First, the classifier — the one model the team does train — needs real shop documents to learn the team's actual four categories in real, messy conditions. Second, evaluation: even the rented pieces need to be tested against real documents before claiming any accuracy number in Chapter 4. No corpus, no evidence for any claim in the report.

SHARP **Why trust "arithmetic checking" as verification — isn't that a narrow definition?**

Yes, and say so plainly. It catches numbers that don't add up internally — quantity × price ≠ stated total. It does *not* catch a misread that stays internally consistent (every related number wrong in the same way). That's a named, accepted limit, not an oversight.

SHARP **How do you protect against prompt injection — text on a document trying to manipulate the LLM?**

Not yet solved, and the honest answer is exactly that. This is a known risk class for any LLM that reads untrusted text, and it isn't specifically defended against in the current design. What partially narrows the risk: schema-constrained extraction forces the LLM's output into fixed fields — it can't return arbitrary free text, only values shaped to the schema. That's not a designed injection defense, but it does limit what a manipulated output could actually do downstream.

---

Do not claim this is solved. A design-stage "we're aware of this, it's flagged for the implementation phase" is a completely acceptable answer here — an overclaim is the actual risk, not the gap itself.

LIKELY **Thirty documents per type — isn't that thin for training a model?**

Fair on its own, but this isn't training from scratch. The classifier fine-tunes an already-pretrained layout-aware model (LayoutLM-family), which already understands general document structure from large-scale pretraining. Thirty is a pilot floor, with collection ongoing — not a hard ceiling.

LIKELY **Why English-only, for a Bangladesh-based audience?**

A real, deliberate scope cut — say it directly, don't dodge it. Reliable OCR and layout recognition for Bangla and handwritten Bangla is a genuinely harder, less mature problem than English. Folding that difficulty into a first version risked the whole timeline. English proves the pipeline design works at all; Bangla is future work, not ruled out permanently.

SHARP **What happens with a document type outside your four categories?**

Not pre-decided — settle this as a team before tomorrow. Two honest directions: either it gets misclassified into the nearest known type (weaker), or the classifier returns a low-confidence result and the record gets flagged rather than forcing a guess (stronger, more consistent with how verification already works). Pick one answer as a team tonight.

LIKELY **No human review step — who checks a flagged record?**

Explicitly the consuming ERP's decision, not this system's. Routing flagged records to a person here would reintroduce the exact manual bottleneck the project exists to remove. This system's job ends at producing a verified, flagged-or-not JSON record.

LIKELY **What accuracy are you actually targeting?**

No measured number yet — nothing is built. The reference point: the LayoutLM-family architecture reports 90–95% document classification accuracy in its own published benchmarks. That's the literature's number on cleaner data, not a promise about this project's result — but it's a grounded floor to target, not an arbitrary guess.

SHARP **Have you measured latency — is this fast enough for someone standing there waiting?**

No real measurement yet. Realistic estimate: OCR plus an LLM call plus verification lands somewhere in single-digit seconds per document — tolerable, but worth watching. If it creeps too high in practice, that's the trigger for revisiting synchronous delivery (open decision #6).

## Before tomorrow — team checklist

- [ ] Everyone can say the six-step pipeline order out loud, unprompted, without looking at notes.
- [ ] Everyone knows which two steps the team builds (classifier, verification) versus which four are rented.
- [ ] Decide as a team: what happens to a document type outside the four categories — say it once, agree on one answer.
- [ ] Decide as a team: how much to say, if anything, about prompt injection — rehearsed verbal answer, or a line added to the deck.
- [ ] Practice saying "nothing is built yet" and "we haven't decided that number yet" out loud — matter-of-fact, not apologetic.
- [ ] Run the two worked-example slides (clean case, flagged case) out loud at least twice each, alone.